



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LAB 05 INSTRUCTIONS

For

Separate path and server delay

Course Code: C1123

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 5.1

Lab 05 — Separate path and server delay

Course: Wireshark Network Analysis Masterclass (C1123)

Topic: 05 — Time Values and Delay Types

Learning outcome: LO2 — Navigate, filter, summarize and time network exchanges.

Time: about 50 minutes

Version: v5.1 · 5 October 2026

Scenario

The /slow page takes almost a second to load. The network team blames the server and the server team blames the network — your timing evidence decides.

Goal

Displayed delta depends on the current filter; capture delta does not.

What you will produce

Timing evidence separating path delay from server delay.

Before you start

- Wireshark 4.6 or later, with the TShark command-line tools on PATH.
- Python 3 for the verification and export scripts (Windows: use `py -3` in place of `python3`).
- Open this lab folder in a terminal; every file the lab needs is inside it.
- Use only the supplied synthetic captures — do not capture on a network you are not authorised to monitor.

Files for this lab

File	What it is for
data/branch-office.pcap	Synthetic branch-office capture used by the lab
assets/scenario.md	The help-desk ticket that sets the scenario
assets/checks.json	Filters and frame counts the fixture must satisfy
assets/expected-evidence.png	The packet list your filter should produce
scripts/verify.py	Checks the capture facts with TShark
scripts/export_evidence.py	Exports this lab's evidence rows to outputs/evidence.csv
outputs/findings.md	Findings sheet you complete

Lab at a glance

1. Measure SYN→SYN/ACK and initial RTT
2. Time the /slow request to its response
3. Compare captured vs displayed deltas
4. Write a hypothesis and next capture point

Step-by-step

1. Prepare your lab workspace. Open this lab folder in a terminal. Windows uses `py -3` in place of `python3`. Wireshark GUI alone is sufficient for the investigation; CLI scripts require TShark on PATH. Run the fixture verification first.

```
python3 scripts/verify.py
```

2. Open `data/branch-office.pcap`. Apply `tcp.stream == 1`. Expand TCP in the final handshake ACK and inspect `tcp.analysis.initial_rtt`: 0.040 seconds. Separately measure SYN to SYN/ACK by setting a time reference on the SYN: 0.030 seconds.

3. Locate GET `/slow` and its HTTP response. Set a new time reference on the request. Record the response delta: 0.760 seconds, including 0.010 seconds to the server ACK and 0.750 seconds thereafter.

4. Apply `http.request.uri == "/slow" || http.response.code == 200` and compare `frame.time_delta` with `frame.time_delta_displayed` on the filtered list. Explain why a displayed gap can span hidden packets.

5. Write a hypothesis in `outputs/findings.md`: slow response with a synthetic 30 ms SYN-to-SYN/ACK interval and 40 ms complete handshake. Name a second observation point required to distinguish processing from a later path delay.

6. Export a reproducible packet table. Record the filter, frame numbers, measurement, explanation and limitation in `outputs/findings.md`.

```
python3 scripts/export_evidence.py
```

Expected evidence

With the lab filter applied, your packet list should match the frames below (produced by TShark from this lab's own capture).

http.request.uri == "/slow" http.response.code == 200					
No.	Time	Source	Destination	Protocol	Info
23	1.140000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
30	1.230000	192.0.2.10	192.0.2.20	HTTP	GET /slow HTTP/1.1
32	1.990000	192.0.2.20	192.0.2.10	HTTP	HTTP/1.1 200 OK (text/plain)
branch-office.pcap		Displayed: 3			

Lab 05 expected evidence

Test it

The supplied `http.request.uri == "/slow" || http.response.code == 200` expression matches 3 frame(s) in the specified capture. Run `scripts/verify.py` and compare the listed frame numbers. Keep your findings and exported table in `outputs/`. Explain the observed result rather than only copying a count.

Troubleshooting

- TShark not found: install Wireshark CLI tools and add the installation folder to PATH; on Windows use the Wireshark install directory.

- A filter returns zero: clear other filters, use the specified capture, and check the expression is in the display toolbar.
- TLS remains opaque: select the matching lab-tls.keys file by absolute path, reload, and remove a key-log preference from another lab.

Try it with TShark

The same evidence from the command line — run it from this lab folder:

```
tshark -n -r data/branch-office.pcap -Y "tcp.stream == 1 && http.time" -T fields -e frame.number -e tcp.analysis.initial_rtt -e http.time
```

Challenge

Develop and explain an alternative filter.

Reflection

Which second observation point would strengthen your conclusion?

Extension (optional)

TCP lab — measure the initial RTT and response times in the TCP trace supplied with the Kurose & Ross labs. Source: J.F. Kurose and K.W. Ross, Wireshark Labs (gaia.cs.umass.edu/kurose_ross/wireshark.php).

Reset

Clear all display filters and return to the C1123-Analyst or Default profile. If you loaded a TLS key log, remove it from Preferences > Protocols > TLS. The supplied captures never need regenerating for the core lab.

Note: The same steps appear in the Learner Guide. The slides show only the scenario and a summary.